

Nestle HR Policy Assistant

RAG Chatbot — Project Summary & Interview Preparation Guide

Ajaykanna E | Simplilearn AI Engineer Master's Program | 2026

1. PROJECT OVERVIEW

This project builds an end-to-end Retrieval-Augmented Generation (RAG) application that allows employees to query Nestle's HR policies through a conversational chatbot interface. Instead of searching through lengthy policy documents manually, employees can ask natural language questions and receive accurate, document-grounded answers instantly.

The system combines document understanding, semantic search, large language model generation, and a deployed user interface — covering the complete GenAI engineering workflow from raw PDF to interactive application.

2. TECH STACK AT A GLANCE

Language	Python 3.x
LLM	OpenAI GPT-3.5 Turbo
Embeddings	OpenAI text-embedding-ada-002
Vector Store	FAISS (Facebook AI Similarity Search)
RAG Framework	LangChain
Document Loader	PyPDFLoader
Text Splitter	RecursiveCharacterTextSplitter
Memory	ConversationBufferMemory
UI Deployment	Gradio Blocks

3. PIPELINE ARCHITECTURE — STEP BY STEP

Understanding each step of the pipeline is essential for interviews. Here is the complete flow with the reasoning behind each decision.

Step 1 — Document Ingestion

PyPDFLoader reads the Nestle HR Policy PDF and extracts text from each page, returning a list of Document objects with page content and metadata (including page numbers for citation).

- Why PDF? HR policies are distributed as PDF documents in enterprise environments.
- Metadata like page number enables source citation in responses.

Step 2 — Text Chunking

RecursiveCharacterTextSplitter splits the full document into smaller overlapping chunks. It tries to split on paragraph boundaries first, then sentences, then words — preserving natural language structure.

- Chunks must fit within the LLM context window.
- Overlap ensures that information spanning a boundary is not lost.
- Recursive splitting preserves semantic coherence better than fixed-character splitting.

Step 3 — Embedding Generation

Each chunk is converted into a numerical vector (embedding) using OpenAI's text-embedding-ada-002 model. These embeddings capture the semantic meaning of the text — similar chunks have embeddings that are close in vector space.

- Embeddings enable semantic search — finding relevant content even when exact words differ.
- ada-002 is cost-effective and produces high-quality embeddings for retrieval tasks.

Step 4 — FAISS Vector Store

All chunk embeddings are stored in a FAISS index. FAISS is an efficient library for similarity search in high-dimensional spaces. At query time, the query is also embedded and FAISS retrieves the top-k most similar chunks using cosine similarity.

- k=4 was chosen to balance context richness with prompt length.
- FAISS is fast and runs locally — no external database needed.

Step 5 — Prompt Engineering

A custom PromptTemplate defines the instructions given to the LLM. It includes: the retrieved context chunks, the conversation history, the user question, and system instructions to answer only from the context and maintain a professional HR tone.

- The system prompt prevents hallucination by instructing the model to say 'I don't know' if the answer is not in the context.
- Professional tone instruction ensures responses are appropriate for enterprise HR use.

Step 6 — Memory Management

ConversationBufferMemory stores the full conversation history. Every user message and bot response is retained and included in subsequent prompts, enabling multi-turn dialogue where follow-up questions are understood in context.

Step 7 — LLM Generation

GPT-3.5 Turbo receives the combined prompt (context + history + question + instructions) and generates a grounded response. Temperature is set to 0.2 for factual consistency.

- Temperature 0.2 — low randomness for factual, consistent policy answers.
- GPT-3.5 Turbo balances quality and cost for enterprise Q&A tasks.

Step 8 — Gradio UI Deployment

A Gradio Blocks interface provides the chatbot UI with a message input, chat history display, example question buttons, and a clear conversation control. The interface runs from the notebook and generates a shareable link.

4. KEY CONCEPTS TO KNOW

RAG vs Fine-tuning

Fine-tuning retrains the model weights on domain-specific data. It is expensive, requires significant data, and becomes outdated when the source document changes. RAG retrieves fresh context at query time from the actual document — no retraining needed. For HR policies that update periodically, RAG is clearly superior.

Semantic Search vs Keyword Search

Keyword search (BM25) matches exact words. Semantic search finds conceptually similar content using embeddings. For natural language HR queries, semantic search handles phrasing variations better. A production system might combine both approaches (hybrid retrieval) for optimal performance.

Why Chunk Overlap Matters

If a critical sentence falls exactly at the boundary between two chunks, neither chunk alone captures the full meaning. Overlap ensures the boundary region appears in both adjacent chunks, so retrieval is less likely to miss important context due to an arbitrary split point.

Context Window Limitation

LLMs have a maximum token limit per request. This is why the full document cannot be passed directly — it must be chunked. Only the most relevant chunks (k=4) are selected and passed as context, keeping the total prompt within the model's token limit.

5. PROJECT LIMITATIONS & IMPROVEMENTS

- ConversationBufferMemory stores full history — long conversations will eventually exceed the context window. ConversationSummaryMemory would be more scalable.
- Fixed chunk size may split tables or numbered policy lists poorly — semantic chunking would preserve document structure better.
- Pure semantic search may miss specific policy codes or section numbers — hybrid retrieval combining FAISS with BM25 would improve precision.
- FAISS index rebuilds on every run — persisting the index to disk would improve startup performance.
- No authentication layer — a production deployment would require employee login and access control.

6. INTERVIEW QUESTIONS AND ANSWERS

Questions are categorised by difficulty. Basic questions test conceptual understanding. Intermediate questions test pipeline knowledge. Advanced questions test critical thinking and production readiness.

Basic — Conceptual Understanding

[Basic] What is RAG and why did you use it instead of fine-tuning?
Answer: RAG stands for Retrieval-Augmented Generation. Instead of retraining the model on HR policy data, RAG retrieves relevant document chunks at query time and feeds them to the LLM as context. I chose RAG because HR policies change frequently — fine-tuning would require retraining every time the document updates. RAG is dynamic, cost-effective, and keeps responses grounded in the actual document.
[Basic] What is a vector store and what role does FAISS play in your project?
Answer: A vector store is a database that stores numerical representations (embeddings) of text. FAISS (Facebook AI Similarity Search) is the vector store I used. When a user asks a question, the query is converted into an embedding and FAISS searches for the top-k most similar document chunks using cosine similarity. In my project, k=4, meaning the 4 most relevant policy chunks are retrieved and passed to GPT-3.5.
[Basic] What is an embedding?
Answer: An embedding is a numerical vector that represents the meaning of a piece of text. Similar sentences have embeddings that are close together in vector space. I used OpenAI's text-embedding-ada-002 model to convert both the document chunks and user queries into embeddings, enabling semantic search — finding relevant content even if the exact words don't match.

[Basic] What does PyPDFLoader do in your pipeline?

Answer: PyPDFLoader is a LangChain document loader that reads a PDF file and extracts its text content page by page. It returns a list of Document objects, each containing the text of one page along with metadata like the page number. This is the first step in my RAG pipeline — converting the raw HR policy PDF into text that can be processed further.

[Basic] What is Gradio and why did you use it for deployment?

Answer: Gradio is a Python library that lets you build interactive web UIs for machine learning models with minimal code. I used Gradio Blocks to create the chatbot interface because it supports multi-turn conversation layouts, example input buttons, and a clean chat history display. It runs directly from the notebook and generates a shareable link, making it easy to demo without a full web deployment.

Intermediate — Pipeline Knowledge

[Intermediate] What is text chunking and why does chunk overlap matter?

Answer: Text chunking splits the full document into smaller pieces so they fit within the LLM's context window. I used RecursiveCharacterTextSplitter which splits on paragraph and sentence boundaries first, preserving natural structure. Chunk overlap means consecutive chunks share some text at their boundaries — this ensures that information spanning a boundary is not lost. Without overlap, a sentence split across two chunks could result in neither chunk fully containing the context needed to answer a question.

[Intermediate] What is a PromptTemplate and why did you create a custom one?

Answer: A PromptTemplate defines the structure of the instruction sent to the LLM. I created a custom one to enforce that the chatbot only answers from the retrieved context and maintains a professional HR tone. The template instructs the model: if the answer is not in the provided context, say so rather than hallucinating. This makes the chatbot factually reliable and appropriate for an enterprise HR use case.

[Intermediate] What is ConversationBufferMemory and what problem does it solve?

Answer: ConversationBufferMemory stores the full conversation history — every user message and bot response — and includes it in each new prompt sent to the LLM. Without memory, the chatbot has no context of previous turns, so a follow-up question like 'What about sick leave?' would be interpreted without knowing the user was already discussing annual leave. Memory enables multi-turn dialogue where follow-up questions are understood in context.

[Intermediate] What does temperature=0.2 mean and why did you choose that value?

Answer: Temperature controls the randomness of the LLM's output. A value of 0 means fully deterministic — the model always picks the highest probability token. A value of 1 is more creative and varied. I set temperature to 0.2 because HR policy answers must be factual and consistent. A higher temperature might produce varied or creative responses that deviate from the actual policy, which would be inappropriate in an enterprise HR context.

[Intermediate] How does the retrieval step work in your pipeline end to end?

Answer: When a user submits a query: first the query text is converted into an embedding using OpenAI text-embedding-ada-002. Then FAISS compares this query embedding against all stored document chunk embeddings using cosine similarity. The top 4 most similar chunks are retrieved. These chunks are combined with the conversation history and injected into the custom PromptTemplate. The full prompt is then sent to GPT-3.5 Turbo which generates a grounded response based only on that retrieved context.

Advanced — Critical Thinking

[Advanced] What are the limitations of your current RAG implementation?

Answer: Several limitations exist. First, ConversationBufferMemory stores the full history, which will exceed the context window in very long conversations — a sliding window or summary memory would be more scalable. Second, fixed chunk size may split important tables or numbered policy lists awkwardly. Third, the system relies on semantic similarity alone — if a user asks a very specific policy number like 'section 4.2', keyword search (BM25) would perform better than embeddings. A hybrid retrieval approach would address this.

[Advanced] How would you improve this project for production deployment?

Answer: For production I would: store the FAISS index persistently so it doesn't rebuild on every restart; add an API key management layer using environment variables and secrets management; replace ConversationBufferMemory with ConversationSummaryMemory for scalability; implement hybrid retrieval combining FAISS semantic search with BM25 keyword search; add logging and monitoring for query patterns; containerize with Docker; and deploy on a cloud platform like AWS or Azure with an authentication layer for employee access control.

[Advanced] What is the difference between semantic search and keyword search? Which is better for this use case?

Answer: Keyword search (like BM25) matches exact words — it finds documents containing the query terms. Semantic search converts both query and documents into embeddings and finds conceptually similar content even if the exact words differ. For HR policy queries, semantic search is generally better because employees phrase questions naturally ('Can I take leave for my child's school event?') rather than using exact policy terminology. However, for specific policy references like document numbers or codes, keyword search is more precise. A hybrid approach combining both would be ideal.

[Advanced] How does LangChain help in building this application?

Answer: LangChain provides pre-built abstractions that chain together the components of the RAG pipeline. Without LangChain, I would need to manually handle the document loading, chunking, embedding generation, vector store integration, prompt construction, memory management, and LLM calls separately. LangChain's chain abstractions — particularly the ConversationalRetrievalChain pattern — connect these components with a consistent interface, handle the context injection into prompts automatically, and manage memory integration, significantly reducing the amount of custom code needed.

7. HOW TO EXPLAIN THIS PROJECT IN AN INTERVIEW

When asked 'Tell me about your GenAI project', use this structure:

Opening — The Problem (1 sentence)

'HR policy documents are long and employees struggle to find specific information quickly.'

Solution — What You Built (2 sentences)

'I built a RAG-based chatbot that takes the HR policy PDF, converts it into searchable embeddings stored in FAISS, and uses GPT-3.5 Turbo to generate accurate answers from retrieved context. Users interact through a Gradio chatbot interface with full conversation memory.'

Technical Depth — If They Ask More

'The pipeline starts with PyPDFLoader for document ingestion, RecursiveCharacterTextSplitter for chunking with overlap, OpenAI embeddings for semantic representation, FAISS for similarity search, a custom PromptTemplate to prevent hallucination, and ConversationBufferMemory for multi-turn dialogue.'

Why This Approach

'I chose RAG over fine-tuning because HR policies change frequently and RAG retrieves fresh context dynamically without retraining. Temperature was set to 0.2 to ensure factual, consistent responses appropriate for an enterprise HR context.'